**This is the Pre-Published Version.**

The following publication C. Ma, X. Chen, K. C. Tan and J. Wu, "Spatiotemporal Decoupled Learning for Spiking Neural Networks," in IEEE Transactions on Neural Networks and Learning Systems, vol. 37, no. 9, pp. 4597-4611, Sept. 2026 is available at https://doi.org/10.1109/TNNLS.2026.3671461. 

# Spatio-Temporal Decoupled Learning for Spiking Neural Networks

Chenxiang Ma, Xinyi Chen, Kay Chen Tan, *Fellow, IEEE*, and Jibin Wu, *Member, IEEE*

***Abstract*—Spiking neural networks (SNNs) have gained significant attention for their potential to enable energy-efficient artificial intelligence. However, effective and efficient training of SNNs remains an unresolved challenge. While backpropagation through time (BPTT) achieves high accuracy, it incurs substantial memory overhead. In contrast, biologically plausible local learning methods are more memory-efficient but struggle to match the accuracy of BPTT. To bridge this gap, we propose spatio-temporal decouple learning (STDL), a novel training framework that decouples the spatial and temporal dependencies to achieve both high accuracy and training efficiency for SNNs. Specifically, to achieve spatial decoupling, STDL partitions the network into smaller subnetworks, each of which is trained independently using an auxiliary network. To address the decreased synergy among subnetworks resulting from spatial decoupling, STDL constructs each subnetwork's auxiliary network by selecting the largest subset of layers from its subsequent network layers under a memory constraint. Furthermore, STDL decouples dependencies across time steps to enable efficient online learning. Extensive evaluations on seven static and event-based vision datasets demonstrate that STDL consistently outperforms local learning methods and achieves comparable accuracy to the BPTT method with considerably reduced GPU memory cost. Notably, STDL achieves 4× reduced GPU memory than BPTT on the ImageNet dataset. Therefore, this work opens up a promising avenue for memory-efficient SNN training. Code is available at https://github.com/ChenxiangMA/STDL.**

***Index Terms*—Spiking neural networks, biologically inspired learning, local learning, neuromorphic computing.**

## I. Introduction

HUMAN brain is renowned for its remarkable efficiency and versatility in cognitive computing, which has long captivated researchers to unravel its underlying structure and operating mechanisms [1]–[3]. These insights have, in turn, driven the development of brain-inspired artificial intelligence (AI) systems aimed at replicating and harnessing the brain's remarkable information processing capabilities [4]. Among these efforts, spiking neural networks (SNNs) have emerged as a promising approach [5]. SNNs seek to faithfully model the dynamic behaviors of biological neural networks, leveraging the spike-based information processing principles observed in the brain [6]. Unlike traditional artificial neural networks (ANNs) that use analog values [7], SNNs employ binary spikes for information representation. Each spiking neuron has a membrane potential, an internal state that evolves and processes incoming spikes over time. This unique spike-based representation, coupled with rich spatiotemporal neuronal dynamics, enables SNNs to perform efficient event-driven computation, where neurons remain inactive until they receive incoming spikes [6]. Characterized by such event-driven computation as well as inherent sparsity, SNNs can offer remarkable energy efficiency when deployed on neuromorphic chips [8]–[10], thus serving as a compelling solution for energy-efficient AI [11], [12].

However, the training of SNNs is non-trivial. Due to the dynamic nature of SNNs, the backpropagation through time (BPTT) [13], [14] algorithm is commonly adopted for training SNNs. BPTT effectively computes parameter gradients by unfolding the SNN over time and backpropagating global error signals through both network layers and time steps. One major challenge in training SNNs lies in the non-differentiability of the neuronal activation function. This is often solved by using surrogate gradients [15], [16], i.e., replacing the gradient of the non-differentiable activation function with a continuous surrogate function that approximates the true gradient. Furthermore, for BPTT training, gradients cannot be computed until forward propagation across all layers and time steps is completed. Consequently, the intermediate neuronal states must be stored in memory until they are revisited for gradient computation, leading to high memory overhead. This becomes a critical bottleneck when simulating large-scale SNN architectures [17].

The biologically inspired local learning approach is an appealing alternative to circumvent the memory overhead issue in BPTT by breaking spatial and temporal dependencies [18], [19]. It enables the online training of each layer independently, and only requires storing the states for a single layer at each time step, leading to significantly reduced memory usage. The unsupervised spike-timing-dependent plasticity (STDP) [20]–[23] is the most widely adopted local learning rule for SNNs, which modulates synaptic weights based on the relative timings of pre- and post-synaptic spikes. However, SNNs trained with STDP are limited to performing simple tasks [24]. Recently, studies have applied gradient descent to local learning by optimizing each layer with an auxiliary classifier and a supervised local loss function [18], [25]. These supervised local learning methods achieve significantly improved accuracy over STDP. While they suffer from poor scalability, they still represent an important step forward in achieving high accuracy with reduced memory requirements compared to BPTT (see Fig. 1). Therefore, achieving both high accuracy and memory

This work was partially supported by the National Natural Science Foundation of China (Grant No. 62306259), the Research Grants Council of the Hong Kong SAR (Grant No. C5052-23G, PolyU15215623, PolyU15229824, PolyU25216423, PolyU15217424, and SRFS2526-5S04), The Hong Kong Polytechnic University (P0056503, P0058445). *(Corresponding author: Jibin Wu (jibin.wu@polyu.edu.hk).)*

Chenxiang Ma, Xinyi Chen, Jibin Wu and Kay Chen Tan are with the Department of Data Science and Artificial Intelligence, The Hong Kong Polytechnic University, Hong Kong SAR, China.

© 2026 IEEE. Personal use of this material is permitted. Permission from IEEE must be obtained for all other uses, in any current or future media, including reprinting/republishing this material for advertising or promotional purposes, creating new collective works, for resale or redistribution to servers or lists, or reuse of any copyrighted component of this work in other works.

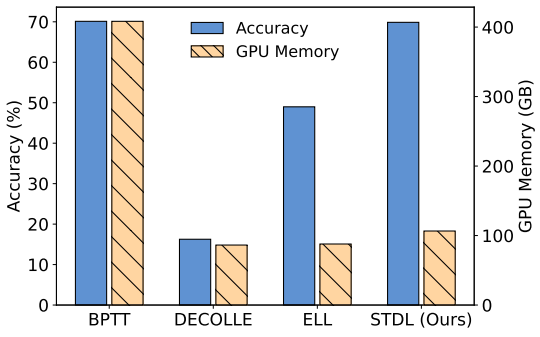



Fig. 1. Comparison of BPTT, supervised local learning rules (DECOLLE and ELL), and the proposed STDL method in terms of accuracy and GPU memory for training the SEWResNet-34 network on the ImageNet dataset.

efficiency in training SNNs remains a critical challenge that requires further research.

In this work, we begin by analyzing supervised local learning methods for training SNNs, and identify that the decoupling of spatial dependencies in these approaches often leads to a weak coupling issue. Specifically, the locally trained layers, optimized solely with their individual objectives, fail to synergize with subsequent layers in achieving the global objective. This represents a major bottleneck that limits the overall network performance. See Section III for more details.

To address this limitation, we propose a novel spatio-temporal decoupled learning (STDL) framework that enables both high-accuracy and memory-efficient training of SNNs. As illustrated in Fig. 2(c), STDL partitions the full network into several subnetworks, each of which is independently trained with an associated auxiliary network, without receiving gradients from subsequent layers. However, increasing the number of subnetworks exacerbates the weak coupling issue. To overcome this, we propose a greedy network partitioning algorithm that constructs a minimal number of subnetworks under a given memory constraint. This is reasonable since layers in typical SNN architectures [26], [27] have different memory requirements, allowing them to be grouped into a single subnetwork without affecting the peak memory usage. Moreover, we theoretically prove that the proposed greedy algorithm yields the optimal subnetwork configuration.

To further enhance the coupling among subnetworks, we design each auxiliary network to guide its associated subnetwork towards producing representations that closely align with those learned via BPTT. In addition, our information-theoretic analysis suggests that enhancing the capacity of the auxiliary network improves the representation alignment. Motivated by these insights, we propose a principled method for constructing auxiliary networks. Specifically, for each subnetwork, we select a subset of its subsequent layers to form an auxiliary network that has the largest capacity while respecting the memory constraint. By combining this construction method with the greedy network partitioning algorithm, STDL enables subnetworks to coordinate effectively toward the global objective, thereby addressing the weak coupling issue. Furthermore, STDL decouples temporal dependencies in the training of each subnetwork by neglecting temporally dependent gradients that contribute negligibly to the overall parameter updates. This ensures that gradients are immediately available at each time step, further improving memory efficiency.

We conduct extensive experiments on image classification datasets, including CIFAR-10 [28], CIFAR-100 [28], SVHN [29], and ImageNet [30], as well as event-based vision datasets, including DVS-CIFAR-10 [31], GAIT-DAY [32], and HAR-DVS [33]. The results consistently demonstrate the superior performance of STDL across these varied task domains. For instance, on the ImageNet dataset using the SEWResNet34 [26] network, STDL outperforms the state-of-the-art supervised local learning methods, deep continuous local learning (DECOLLE) [25] and efficient local learning (ELL) [18], by over 50% and 20% in accuracy, respectively. This significant performance improvement highlights the effectiveness of STDL in enhancing the coordination between the independently trained subnetworks. Notably, STDL achieves comparable accuracy to the BPTT method while reducing the GPU memory usage by a factor of $4.0\times$.

Our key contributions are summarized as follows:

- STDL is the first method that effectively harnesses the memory advantage of local learning while achieving performance on par with BPTT, representing a significant step toward memory-efficient SNN training.
- We propose novel methods for partitioning the full network into subnetworks and constructing auxiliary networks. These novel designs effectively address the weak coupling issue, delivering enhanced performance while retaining the memory efficiency advantage.
- We conduct a comprehensive validation of STDL on seven benchmark datasets, consistently demonstrating remarkable accuracy that is significantly superior to existing local learning rules and on par with BPTT, as well as the substantially reduced GPU memory usage. Our experiments also demonstrate the strong generalization ability of STDL to different spiking neuron models.
- We conduct extensive ablation studies to evaluate the memory efficiency of STDL and to assess the contribution of each individual component. In addition, our representation analysis shows that STDL learns representations closely aligned with those produced by BPTT, offering a compelling explanation for the efficacy of STDL.

The remainder of this work is organized as follows. In Section II, we provide a review of existing SNN training methods. In Section III, we analyze the memory demands of BPTT as well as the reasons behind the inferior accuracy of existing supervised local learning methods, and identify the weak coupling issue as the major bottleneck. Section IV details the proposed STDL method, followed by extensive experimental results and analyses in Section V. Finally, we conclude the paper in Section VI.

## II. Related Works

In this section, we review existing training methods for SNNs.

### A. BPTT Training

Recently, BPTT training for SNNs has been advanced by developing adaptive surrogate gradients [34], [35]. Additionally,

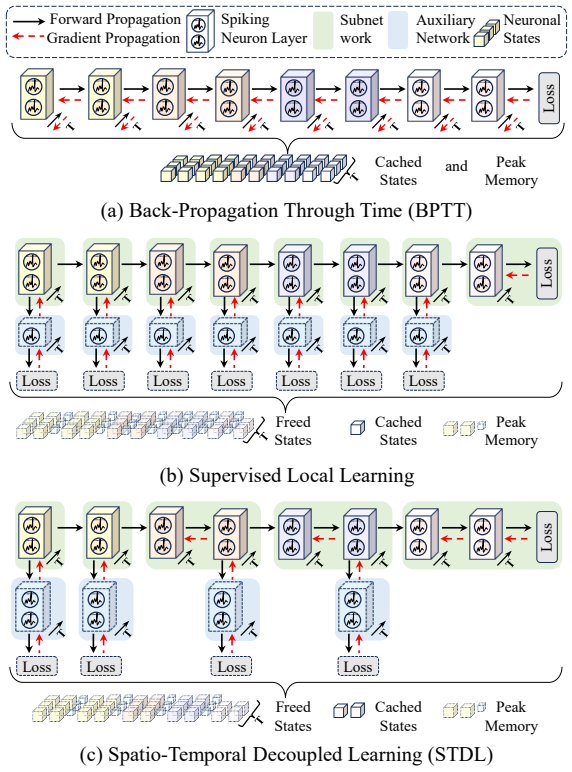



Fig. 2. **Comparison of BPTT, supervised local learning, and our STDL for training SNNs.** (a) BPTT caches the entire trajectory of neuronal states during forward propagation for gradient computation, with memory overhead scaled along with both the number of layers and time steps. (b) Supervised local learning caches the states of only one layer at each time step, as it enables the online learning of each network layer along with an auxiliary linear classifier independently. (c) STDL partitions the original network into subnetworks by grouping as many adjacent layers as possible without affecting the peak memory. Each subnetwork is independently trained with an auxiliary network in an online learning manner. For both (b) and (c), once the forward propagation of a subnetwork and its auxiliary network is completed, their gradients are immediately calculated, followed by freeing the cached states.

efforts have been made to enhance the stability of BPTT. For instance, loss functions [36], [37] are developed to regulate the distribution of membrane potential, facilitating network optimization and convergence. Specialized variants of batch normalization techniques [38] are developed for SNNs to accelerate convergence. Furthermore, trainable decays [39], adaptive thresholds [40], gating mechanisms [41], [42], attention mechanisms [43], heterogeneous synapses [44], [45], hybrid coding [46], augmented spikes [47], [48], neural oscillations [49],and multi-compartmental modeling [50], [51] are explored to augment the capabilities of spiking neuron models, thereby improving BPTT's performance on practical tasks. However, BPTT incurs substantial GPU memory overhead.

### B. Local Learning

Local learning has gained increasing attention due to its appealing characteristics of biological plausibility and memory efficiency [19], [52]. An early attempt [21] to leverage the STDP rule to train single-layer spiking networks effectively enables unsupervised visual feature extraction. Subsequent works extend the approach to train spiking convolutional networks [22] or utilize a symmetric variant of STDP in combination with other bio-plausible mechanisms to improve performance [23]. Despite these advancements, STDP-trained SNNs demonstrate effectiveness primarily on simple tasks, such as MNIST [22], [23], and encounter significant challenges when dealing with more complex tasks. Regarding supervised local learning, DECOLLE [25] employs fixed and random auxiliary classifiers to construct layer-wise loss functions. It also enables continuous training at each time step by ignoring all temporally dependent gradients. ELL [18] further enhances accuracy by replacing the fixed auxiliary classifiers with trainable ones. However, they still have a large accuracy gap compared to BPTT, especially for large-scale SNNs.

In addition, previous efforts have developed hybrid local-global learning methods. In local tandem learning (LTL) [53], local learning is employed to train SNNs assisted by a pre-trained ANN. Local plasticity can also be integrated with top-down supervision to facilitate multi-scale learning [54]. A recent work proposes the spiking time sparse feedback (STSF) method [55] that associates sparse direct feedback alignment with STDP to achieve hybrid learning.

### C. Online Learning

Online learning can be regarded as a simplified variant of local learning with decoupled temporal dependencies only, where parameter gradients are computed in real time. Recent works leverage gradient approximation to achieve the online learning of SNNs. For instance, E-Prop [56] and OTTT [57] approximate the gradients of BPTT by eligibility traces, which are the temporal accumulation of presynaptic activities. SLTT [58] disregards all temporal dependencies in gradient computation, with no need for maintaining additional traces. Another work [59] proposes a specific neuron model and a regularization term to improve online learning, which introduces additional computational overhead, limiting its application on large architectures. Despite these advancements, online learning methods require storing states of all layers, leading to substantial memory usage.

### D. Other End-to-End Learning

Other approaches for end-to-end training of SNNs include the ANN-to-SNN conversion approach [48], [60], which constructs an SNN by converting weights from a pre-trained ANN with a similar structure. Additionally, tandem learning approaches [53], [61], [62] introduce weight sharing between an ANN and an SNN, where the ANN provides gradients to train the SNN. These methods fundamentally differ from ours as they rely on global objectives.

More recently, several approaches have sought to improve training efficiency by modifying temporal computation. Temporal Reversible SNN (T-RevSNN) architecture [63] reduces the cost of temporal processing by disabling the intrinsic temporal dynamics of most neurons and encoding temporal information through reversible interactions among a small subset of neurons. This design enables the memory cost of both training and inference to be constant with respect to the number of time steps. Another line of work [64], [65] proposes using multi-bit spike representations during training while

preserving binary spikes at inference. This strategy enables SNNs to be optimized using only a single time step, thereby substantially reducing training time. Although these methods improve training efficiency, they still operate within a global end-to-end training framework and maintain spatial dependencies across layers. In contrast, our STDL decouples spatial dependencies, providing a fundamentally different approach to reducing memory consumption during training.

## III. Motivation

In this section, we first analyze the substantial memory overhead associated with training an SNN using BPTT. We then elucidate how local learning can alleviate this memory bottleneck. Finally, we examine the reasons for the significant performance gap between local learning methods and BPTT.

### A. Memory Consumption of BPTT

During the forward propagation, input samples traverse the network layer-by-layer until they reach the output layer, where predictions are generated. Let us consider a layer $\ell$ which receives afferent spikes from the preceding layer. These input spikes are integrated into an internal state called the membrane potential. When the membrane potential exceeds a pre-defined threshold, a spike is generated, followed by a reset process. The neuronal dynamics of the commonly-used leaky integrate-and-fire (LIF) neuron model can be represented in the following discrete-time form [14]:

$$\boldsymbol{m}^{\ell}[t] = \lambda \boldsymbol{u}^{\ell}[t-1] + \boldsymbol{W}^{\ell}\boldsymbol{s}^{\ell-1}[t], \tag{1}$$

$$\boldsymbol{s}^{\ell}[t] = \Theta(\boldsymbol{m}^{\ell}[t] - V_{\text{th}}), \tag{2}$$

$$\boldsymbol{u}^{\ell}[t] = \boldsymbol{m}^{\ell}[t] - V_{\text{th}}\boldsymbol{s}^{\ell}[t], \tag{3}$$

$$\Theta(x) = \begin{cases} 1, & x >= 0 \\ 0, & \text{otherwise} \end{cases}, \tag{4}$$

where $\boldsymbol{m}^{\ell}[t]$ and $\boldsymbol{u}^{\ell}[t]$ represent the membrane potentials before firing and after reset, respectively. $\lambda$ denotes a decay factor that controls the decay rate of the membrane potential. $\boldsymbol{s}^{\ell-1}[t]$ represents afferent spikes. $\boldsymbol{W}^{\ell}$ is a weight matrix connecting adjacent layers. The Heaviside function $\Theta(x)$ determines whether a spike is generated based on the relationship between the membrane potential $\boldsymbol{m}^{\ell}[t]$ and the threshold $V_{\text{th}}$.

Upon reaching the output layer, a loss $\mathcal{L}$ is calculated by measuring the discrepancy between the predictions and the targets. To update network parameters, gradients of the loss with respect to each parameter are computed through a backward pass. This pass starts from the last layer and the last time step and propagates in reverse across all network layers and time steps. The gradient of the loss with respect to the weights of layer $\ell$ is formulated as [14]:

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{\ell}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial \boldsymbol{m}^{\ell}[t]} \frac{\partial \boldsymbol{m}^{\ell}[t]}{\partial \boldsymbol{W}^{\ell}} = \sum_{t=1}^{T} \boldsymbol{\delta}^{\ell}[t] \boldsymbol{s}^{\ell-1}[t]^{\top}, \tag{5}$$

$$\boldsymbol{\delta}^{\ell}[t] = \begin{cases} \frac{\partial \mathcal{L}}{\partial \boldsymbol{m}^{L}[T]}, & \ell = L \text{ and } t = T \\ \boldsymbol{\delta}^{L}[t+1] \frac{\partial \boldsymbol{m}^{L}[t+1]}{\partial \boldsymbol{m}^{L}[t]} + \frac{\partial \mathcal{L}}{\partial \boldsymbol{m}^{L}[t]}, & \ell = L \text{ and } t < T \\ \boldsymbol{\delta}^{\ell+1}[T] \frac{\partial \boldsymbol{m}^{\ell+1}[T]}{\partial \boldsymbol{s}^{\ell}[T]} \frac{\partial \boldsymbol{s}^{\ell}[T]}{\partial \boldsymbol{m}^{\ell}[T]}, & \ell < L \text{ and } t = T \\ \boldsymbol{\delta}^{\ell}[t+1] \frac{\partial \boldsymbol{m}^{\ell}[t+1]}{\partial \boldsymbol{m}^{\ell}[t]} + \boldsymbol{\delta}^{\ell+1}[t] \frac{\partial \boldsymbol{m}^{\ell+1}[t]}{\partial \boldsymbol{m}^{\ell}[t]}, & \text{otherwise} \end{cases} \tag{6}$$

where $\boldsymbol{\delta}^{\ell}[t] \triangleq \frac{\partial \mathcal{L}}{\partial \boldsymbol{m}^{\ell}[t]}$, and $T$ and $L$ denote the number of time steps and layers, respectively. The derivative of the Heaviside function, i.e., $\frac{\partial \boldsymbol{s}_i^{\ell}[t]}{\partial \boldsymbol{m}_i^{\ell}[t]}$, is zero for all values of $\boldsymbol{m}_i^{\ell}[t]$ except for the case $\boldsymbol{m}_i^{\ell}[t] = V_{\text{th}}$, where it is infinite. Surrogate functions [15] are employed to handle this non-differentiability, i.e., $\mathbb{H}\left(\boldsymbol{m}_i^{\ell}[t]\right) \approx \frac{\partial \boldsymbol{s}_i^{\ell}[t]}{\partial \boldsymbol{m}_i^{\ell}[t]}$, such as the triangle function:

$$\mathbb{H}\left(\boldsymbol{m}_i^{\ell}[t]\right) = \frac{1}{\gamma^2} \max\left(0,\ \gamma - |\boldsymbol{m}_i^{\ell}[t] - V_{\text{th}}|\right), \tag{7}$$

where $\gamma$ is a hyperparameter typically set to 1.

According to Eq. (6), BPTT needs to access the gradients from subsequent layers and time steps to compute the gradients for the current layer at the current time step. Due to such dependencies across spatial and temporal dimensions, gradient computation in BPTT can only start after the full forward pass is completed. Consequently, all the intermediate neuronal states necessary to compute gradients are stored in memory during the forward pass. The resultant memory consumption is proportional to the total number of layers and time steps (see Fig. 2 (a) for the illustration). Specifically, according to Eq. (5), afferent spikes $\boldsymbol{s}^{\ell-1}[t]$ are essential for calculating the weight gradients. Therefore, all afferent spikes across both spatial and temporal dimensions, i.e., $\left\{\boldsymbol{s}^{\ell-1}[t] \,|\, \ell \in \{1,\ldots,L\}, t \in \{1,\ldots,T\}\right\}$, must be stored in memory during the forward pass. Similarly, all membrane potentials, i.e., $\left\{\boldsymbol{m}^{\ell}[t] \,|\, \ell \in \{1,\ldots,L\}, t \in \{1,\ldots,T\}\right\}$, need to be retained in memory to compute the surrogate gradient $\mathbb{H}\left(\boldsymbol{m}_i^{\ell}[t]\right)$ according to Eq. (7). The memory consumption becomes substantial when training large-scale spiking models with a long time duration.

### B. Memory-Efficient Training via Decoupling Spatial and Temporal Dependencies

As aforementioned, the substantial memory requirement of BPTT stems from the spatial and temporal dependencies. Hence, a natural solution for memory-efficient training is to decouple these dependencies, allowing gradient computations to occur before the entire forward pass is completed. In this way, cached states can be released from memory, leading to a reduction in the memory requirement. This insight aligns with the principles of synaptic plasticity observed by neuroscientists [20], [66], where synaptic connections are adjusted locally without waiting for signals from distant and unconnected layers. Such local learning rules have been successfully applied in training SNNs. Among them, supervised local learning methods [18], [25] have demonstrated superior performance on practical tasks. As illustrated in Fig. 2 (b), these methods enable the online learning of each layer independently using a layer-wise auxiliary classifier, requiring only the states necessary for updating a single layer to be retained in memory at the current time step. However, despite their high memory

TABLE I
PERFORMANCE OF SUPERVISED LOCAL LEARNING FOR TRAINING RESNET-18 ON CIFAR-10 ($T = 1$), WHICH IS EVENLY DIVIDED INTO $K$ INDEPENDENTLY TRAINED SUBNETWORKS. "$K = 1$" IS BPTT TRAINING.

| | $K$=1 | $K$=3 | $K$=5 | $K$=7 | $K$=9 |
|---|---|---|---|---|---|
| Acc. (%) | 94.01 | 91.79 | 90.71 | 85.88 | 85.53 |
| Mem. (GB) | 4.56 | 3.36 | 2.63 | 2.29 | 2.29 |

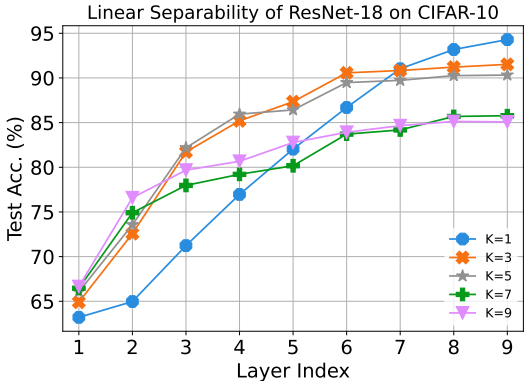



Fig. 3. Comparison of layer-wise linear separability among supervised local learning variants that evenly divide ResNet-18 into $K$ subnetworks.

efficiency, they still face a significant accuracy gap compared to BPTT, especially in large-scale SNNs [18].

### C. The Weak Coupling Issue in Local Learning

Recent studies on online learning have demonstrated that decoupling temporal dependencies, i.e., training each time step independently, has a negligible impact on model accuracy [57], [58]. This is because the gradients associated with temporal dependencies contribute minimally to the overall parameter gradients [58]. Motivated by this insight, we hypothesize that the primary cause of the performance gap between supervised local learning and BPTT lies in the decoupling of spatial dependencies. We are thus motivated to address the accuracy loss resulting from the spatial decoupling.

A straightforward solution to mitigate this issue is to reduce the degree of independence across the spatial dimension by decreasing the number of locally trained layers. However, a naive network partitioning strategy, where the network is evenly divided into several subnetworks, each comprising an equal number of consecutive layers, fails to match the accuracy of BPTT and offers only marginal memory savings. This insight is supported by empirical results. Specifically, we evaluate supervised local learning on CIFAR-10 using ResNet-18, which is evenly partitioned into $K$ subnetworks, each trained independently with a trainable linear classifier [18]. As shown in Table I, while reducing the number of subnetworks narrows the accuracy gap to BPTT, the gap remains substantial. Moreover, this improvement comes at the cost of significantly reduced memory efficiency. These findings highlight that achieving a comparable accuracy to BPTT while retaining the memory efficiency of local learning is non-trivial.

To investigate the underlying reasons for the performance degradation, we analyze the linear separability of each layer's representation learned using these local learning variants and compare it with that obtained via BPTT. As shown in Fig. 3, local learning variants exhibit distinct patterns of linear separability. Specifically, they tend to learn more linearly separable representations in early layers, but show reduced separability in deeper layers, compared to BPTT. This discrepancy becomes increasingly pronounced as the number of independently trained subnetworks increases. This observation indicates that early layers, optimized solely for their local objectives, fail to produce representations that are well aligned with the needs of subsequent layers and benefit the global objective. We refer to this as *the weak coupling issue*, where independently trained subnetworks fail to collaborate effectively toward the global objective, which is the major reason for the suboptimal performance of local learning methods.

## IV. SPATIO-TEMPORAL DECOUPLED LEARNING (STDL)

In this section, we propose STDL, a novel memory-efficient training method for SNNs. We first present an overview of STDL, and then describe in detail how it mitigates the weak coupling issue through a greedy network partitioning method and an information-theoretically guided auxiliary network construction approach. Lastly, we explain how STDL incorporates temporal decoupling to further reduce memory overhead without compromising performance.

### A. Overview of STDL

As illustrated in Fig. 2(c), STDL enables memory-efficient training of SNNs by decoupling both spatial and temporal dependencies. To decouple spatial dependencies, STDL partitions the full network into several smaller subnetworks, each of which is trained independently with the supervision of its associated auxiliary network. To address the weak coupling issue, STDL introduces a provably optimal greedy network partitioning algorithm that minimizes the number of subnetworks under a given memory constraint. Additionally, STDL incorporates an information-theoretically grounded auxiliary network construction method that guides each subnetwork to produce representations that closely resemble those learned through BPTT, thereby enhancing the coupling across subnetworks. STDL further decouples temporal dependencies by discarding uninformative temporally dependent gradients, allowing parameter gradients to be computed at each time step.

Formally, consider an $L$-layer SNN denoted as $\mathcal{F} : \mathcal{X} \to \mathcal{Y}$. A mini-batch of input samples $\boldsymbol{x} \in \mathcal{X}$ is processed sequentially through the network layers as:

$$\mathcal{F}(\boldsymbol{x}) = f^L \circ \cdots \circ f^\ell \circ \cdots \circ f^1(\boldsymbol{x}), \tag{8}$$

where $f^\ell(\cdot)$ signifies the operation of the $\ell$-th layer, and $\circ$ is the function composition operator. By the greedy network partitioning algorithm detailed in Section IV-B, STDL partitions the full network $\mathcal{F}$ into $K$ subnetworks $(K \leq L)$ as:

$$\mathcal{F}(\boldsymbol{x}) = \mathcal{F}^K \circ \cdots \circ \mathcal{F}^k \circ \cdots \circ \mathcal{F}^1(\boldsymbol{x}). \tag{9}$$

The propagation of gradients between adjacent subnetworks is prohibited by using the stop-gradient operator $\mathrm{sg}(\cdot)$ to decouple their spatial dependency during training:

$$\boldsymbol{s}^k = \mathcal{F}^k\left(\mathrm{sg}(\boldsymbol{s}^{k-1})\right). \tag{10}$$

Each subnetwork $\mathcal{F}^k$ is paired with an auxiliary network $\mathcal{G}^k : \mathcal{H} \to \mathcal{Y}$, which maps its spike-based representation $\boldsymbol{s}^k \in \mathcal{H}$ to a prediction. The construction of $\mathcal{G}^k$ is described in Section IV-C. The pair $(\mathcal{F}^k, \mathcal{G}^k)$ is jointly optimized using a local loss function $\mathcal{L}^k(\mathcal{G}^k(\boldsymbol{s}^k), \boldsymbol{y})$ that compares the local prediction with the target $\boldsymbol{y} \in \mathcal{Y}$. This loss is computed as the sum of instantaneous losses over the time window:

$$\mathcal{L}^k\left(\mathcal{G}^k(\boldsymbol{s}^k), \boldsymbol{y}\right) = \sum_{t=1}^{T} \mathcal{L}^k[t]\left(\mathcal{G}^k(\boldsymbol{s}^k[t]), \boldsymbol{y}[t]\right). \tag{11}$$

To decouple temporal dependencies, STDL enables the online learning of each subnetwork and its auxiliary network, as detailed in Section IV-D. Note that the last subnetwork $\mathcal{F}^K$ is not paired with an auxiliary network, as it already includes the output classifier. All auxiliary networks are used only during training and discarded during inference.

### *B. Greedy Network Partitioning for a Minimal Number of Subnetworks*

As shown in Section III-C, increasing the number of subnetworks exacerbates the weak coupling issue and leads to noticeable accuracy degradation. Additionally, due to different memory requirements across layers in SNN architectures [26], [27], it is often possible to group several successive layers into a single subnetwork without affecting the peak memory footprint. These insights motivate us to minimize the number of subnetworks under a given memory budget.

To formalize this objective, let $\boldsymbol{P} = \{p_1, p_2, \ldots, p_K\}$ denote the set of subnetwork boundary indices, where each $p_k$ corresponds to the index of the last layer in the $k$-th subnetwork, and $K$ is the total number of subnetworks. Let $\mathrm{mem}_\ell$ represent the memory footprint of the $\ell$-th layer, and let $M_{\mathrm{aux}}$ be the maximum allowable memory footprint per subnetwork. The objective is to partition the $L$-layer network into the smallest number of subnetworks such that the memory footprint of each subnetwork does not exceed $M_{\mathrm{aux}}$. This problem can be formally defined as:

$$\begin{aligned} &\min_{\boldsymbol{P}} \quad K \\ \text{subject to} &\sum_{\ell=p_{k-1}+1}^{p_k} \mathrm{mem}_\ell \le M_{\mathrm{aux}}, \quad k = 1, \ldots, K, \\ &1 \le p_1 < p_2 < \cdots < p_K = L. \end{aligned} \tag{12}$$

Solving this problem via brute force is computationally intractable, as it requires evaluating all possible partitions of the $L$ layers, resulting in exponential complexity. To address this, we propose a greedy network partitioning algorithm that efficiently constructs the optimal solution in a single pass.

Specifically, the greedy algorithm initializes at the layer following the previous subnetwork, and incrementally adds layers into the current subnetwork until the total memory footprint of the included layers surpasses the given memory constraint. Any remaining layers are then assigned to the next subnetwork with the same iterative process. This process continues until all layers are allocated into subnetworks. The pseudocode for the greedy algorithm is provided in Algorithm 1. The following proposition establishes the optimality of this greedy algorithm, and the proof can be found in Appendix A.

**Algorithm 1:** Greedy Network Partitioning

**Input:** Memory footprint of each layer $\{\mathrm{mem}_\ell\}_{\ell=1}^{L}$, total number of layers $L$, memory constraint per subnetwork $M_{\mathrm{sub}}$
**Output:** Subnetwork boundaries $\boldsymbol{P} = \{p_1, p_2, \ldots, p_K\}$

```
P ← ∅;
current_mem ← 0;
for ℓ = 1 to L do
    current_mem ← current_mem + mem_ℓ;
    if current_mem > M_sub then
        p ← ℓ − 1;
        P ← P ∪ {p};
        current_mem ← mem_ℓ;
P ← P ∪ {L};
return P
```

**Proposition 1.** *The greedy network partitioning algorithm described in Algorithm 1 yields an optimal solution to the optimization problem defined in Eq. 12.*

### *C. Building Auxiliary Network for Representation Alignment*

The design of auxiliary networks is crucial for partitioned subnetworks to contribute collaboratively to the global objective. To promote the coupling among subnetworks, we aim to construct auxiliary networks that enable the representations of their subnetworks to align with those produced by BPTT. This is reasonable as the subnetworks trained using BPTT are naturally coupled. To achieve this, we introduce a structure prior into the auxiliary networks. Specifically, for a subnetwork, we select a subset of its subsequent layers as its auxiliary network. In this way, the resultant auxiliary network can incorporate critical transformations in the subsequent layers, thereby enabling the subnetwork's representation to be similar to that of BPTT. However, under a given memory constraint, it remains non-trivial to determine which subsequent layers should be selected to construct the auxiliary network. To address this challenge, we analyze the discrepancy between representations produced by the auxiliary network and BPTT from an information-theoretic perspective [67]. Building on this analysis, we propose to select the subsequent layers that maximize the auxiliary network's capacity, i.e., depth and width, within the memory budget, as explained below.

Without loss of generality, we consider the $k$-th subnetwork. Let $\boldsymbol{s}^k_{\mathrm{local}}$ denote the spike-based representation learned by its auxiliary network $\mathcal{G}^k$, and let $\hat{\boldsymbol{y}}^k_{\mathrm{local}}$ denote the local prediction generated by $\mathcal{G}^k$. Similarly, we define $\boldsymbol{s}^k_{\mathrm{global}}$ as the representation learned via BPTT and $\hat{\boldsymbol{y}}_{\mathrm{global}}$ as the global prediction. Then, we quantify the difference between $\boldsymbol{s}^k_{\mathrm{local}}$ and $\boldsymbol{s}^k_{\mathrm{global}}$ from an information-theoretic perspective. Specifically, we regard $\boldsymbol{s}^k_{\mathrm{local}}$, $\boldsymbol{s}^k_{\mathrm{global}}$, $\hat{\boldsymbol{y}}^k_{\mathrm{local}}$, $\hat{\boldsymbol{y}}_{\mathrm{global}}$, along with the input $\boldsymbol{x}$ and the label $\boldsymbol{y}$, as random variables. The information encoded by $\boldsymbol{s}^k_{\mathrm{local}}$ and $\boldsymbol{s}^k_{\mathrm{global}}$ is then characterized using two

mutual information quantities [67]: $I(\boldsymbol{s}^k;\boldsymbol{x})$, quantifying the information related to the input $\boldsymbol{x}$, and $I(\boldsymbol{s}^k;\boldsymbol{y})$, quantifying the information related to the label $\boldsymbol{y}$. In the following analysis, we focus exclusively on the discrepancy between $I(\boldsymbol{s}^k_{\text{local}};\boldsymbol{y})$ and $I(\boldsymbol{s}^k_{\text{global}};\boldsymbol{y})$. The discrepancy between $I(\boldsymbol{s}^k_{\text{local}};\boldsymbol{x})$ and $I(\boldsymbol{s}^k_{\text{global}};\boldsymbol{x})$ can be derived using an analogous procedure.

Our objective is to find an auxiliary network $\mathcal{G}^k$ that can guide its subnetwork to generate $\boldsymbol{s}^k_{\text{local}}$ closely resembling $\boldsymbol{s}^k_{\text{global}}$ under the memory constraint $M_{\text{sub}}$. This can be formulated by minimizing the difference between the two mutual information quantities:

$$\begin{aligned} \min_{\mathcal{G}^k} \quad & |I(\boldsymbol{s}^k_{\text{global}};\boldsymbol{y}) - I(\boldsymbol{s}^k_{\text{local}};\boldsymbol{y})| \\ \text{subject to} \quad & M_{\mathcal{G}^k} \le M_{\text{sub}}, \end{aligned} \tag{13}$$

where $M_{\mathcal{G}^k}$ denotes the memory footprint of the auxiliary network $\mathcal{G}^k$. Note that $I(\boldsymbol{s}^k_{\text{local}};\boldsymbol{y}) \le I(\boldsymbol{s}^k_{\text{global}};\boldsymbol{y})$, where the equality holds if and only if the auxiliary network is exactly equivalent to the full subsequent layers. However, due to the memory constraint, auxiliary networks typically possess lower capacity, and thus inherently retain less information about $\boldsymbol{y}$. Since the subsequent layers of the $k$-th subnetwork are fixed, $I(\boldsymbol{s}^k_{\text{global}};\boldsymbol{y})$ is constant. Consequently, minimizing Eq. 13 is equivalent to maximizing $I(\boldsymbol{s}^k_{\text{local}};\boldsymbol{y})$. Instead of directly analyzing the maximization, we seek to maximize a surrogate objective, specifically a lower bound on the mutual information $I(\boldsymbol{s}^k_{\text{local}};\boldsymbol{y})$. We achieve this by leveraging the data processing inequality (DPI) [67]. Specifically, with the Markov chain $\boldsymbol{y} \to \boldsymbol{x} \to \boldsymbol{s}^k \to \hat{\boldsymbol{y}}$, the DPI guarantees:

$$I(\boldsymbol{s}^k_{\text{local}};\boldsymbol{y}) \ge I(\hat{\boldsymbol{y}}^k_{\text{local}};\boldsymbol{y}). \tag{14}$$

This suggests that the prediction $\hat{\boldsymbol{y}}$, being a function of the representation $\boldsymbol{s}^k$, cannot contain more information about $\boldsymbol{y}$ than the representation itself. As a result, minimizing the mutual information gap in Eq. 13 is equivalent to maximizing the lower bound $I(\hat{\boldsymbol{y}}^k_{\text{local}};\boldsymbol{y})$. The following proposition establishes how this quantity can be increased by expanding the expressive capacity of the auxiliary network.

**Proposition 2.** *Let $\mathcal{G}^k_{\boldsymbol{\theta}}$ be the $k$-th auxiliary network, parameterized by $\boldsymbol{\theta} \in \mathbb{R}^{D\times W}$ with depth $D$ and width $W$, and assume it is sufficiently trained without significant overfitting or underfitting. Let $\hat{\boldsymbol{y}}^k_{\text{local}} = \mathcal{G}^k_{\boldsymbol{\theta}}(\boldsymbol{s}^k)$ denote its prediction given input $\boldsymbol{s}^k$. Consider an expanded auxiliary network $\mathcal{G}^k_{\boldsymbol{\theta}'}$ with parameters $\boldsymbol{\theta}' \in \mathbb{R}^{D'\times W'}$, satisfying $D' \ge D$ and $W' \ge W$. Assume $\mathcal{G}^k_{\boldsymbol{\theta}'}$ is also sufficiently trained, and let $\hat{\boldsymbol{y}}^{k'}_{\text{local}} = \mathcal{G}^k_{\boldsymbol{\theta}'}(\boldsymbol{s}^k)$ denote its prediction. Then, the mutual information between the predictions and the label satisfies:*

$$I\left(\hat{\boldsymbol{y}}^k_{\text{local}};\boldsymbol{y}\right) \le I\left(\hat{\boldsymbol{y}}^{k'}_{\text{local}};\boldsymbol{y}\right). \tag{15}$$

Appendix B provides the proof. Proposition 2 establishes that the mutual information $I(\hat{\boldsymbol{y}}^k_{\text{local}};\boldsymbol{y})$ is monotonically non-decreasing with respect to the depth and width of the auxiliary network. Motivated by this insight, we design each auxiliary network to maximize its expressive capacity, i.e., its depth and width, within the memory budget $M_{\text{sub}}$. Since increasing network depth yields exponentially greater expressive capacity than increasing width [7], we prioritize depth expansion over width. An illustration of our method is shown in Fig. 4.

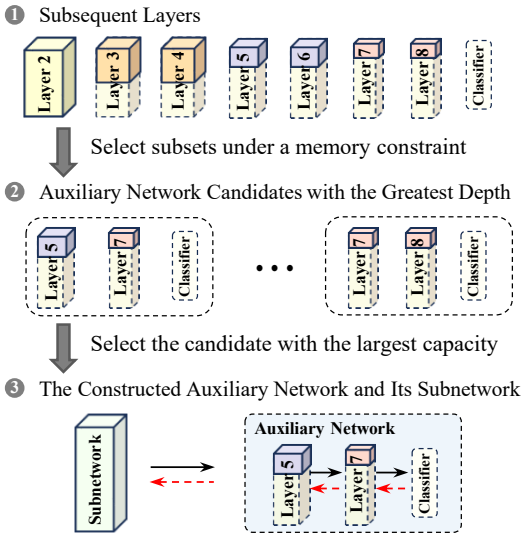



Fig. 4. **Illustration of auxiliary network construction in STDL.** For each subnetwork, the auxiliary network is constructed by selecting a subset of its subsequent layers. The subnet has the largest expressive capacity, i.e., depth and width, while not exceeding a given memory constraint. The shaded area within each layer represents its memory footprint during training.

Concretely, we first determine the maximum number of layers from the subsequent layers without violating the memory constraint. We then construct the auxiliary network by selecting layers accordingly to match this maximum allowable depth. Among candidate configurations with the same depth, we prioritize the widest architecture, which has the greatest total number of channels across all auxiliary layers, as it offers greater expressive capacity within the same depth budget. Additionally, following conventions in modern neural network design [26], [27], if an auxiliary layer expands its number of channels relative to its preceding layer, average pooling is applied to downsample the spatial resolution proportionally. Each auxiliary network is completed by appending an adaptive average pooling layer to aggregate spatial information, followed by a linear classifier that projects the pooled representation into the target space. Note that the auxiliary networks do not share parameters with subsequent layers and operate independently of them. There is neither gradient flow nor state reuse between any auxiliary network and the subsequent layers.

### *D. Decoupling Temporal Dependencies by Omitting Temporally Dependent Gradients*

While auxiliary networks allow each subnetwork to be trained independently, the computation of parameter gradients within each subnetwork still depends on future time steps due to temporal dependencies. As a result, gradients cannot be computed immediately at the current time step, and the memory footprint remains proportional to the total number of time steps. To address this, STDL explicitly discards temporally dependent gradients that contribute negligibly to the overall parameter update. By ignoring these low-impact temporal gradients, STDL enables the online learning of each subnetwork and its auxiliary network, thereby further reducing

TABLE II
COMPARISON OF STDL AND OTHER LEARNING METHODS ON FOUR IMAGE CLASSIFICATION DATASETS. AVERAGED ACCURACY AND STANDARD DEVIATION ARE REPORTED FROM THREE INDEPENDENT TRIALS. THE GPU MEMORY FOOTPRINT IS MEASURED WITH A BATCH SIZE OF 512.

| Method | Decoupled in | | CIFAR-10 | | CIFAR-100 | | SVHN | | ImageNet | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Space | Time | ResNet-18 (T=4) | | ResNet-19 (T=4) | | VGG16 (T=4) | | SEWResNet-34 (T=4) | |
| | | | Acc. (%) | Mem. (GB) | Acc. (%) | Mem. (GB) | Acc. (%) | Mem. (GB) | Acc. (%) | Mem. (GB) |
| BPTT [14] | ✗ | ✗ | 95.12+0.15 | 15.45 | 78.83±0.10 | 39.44 | 96.09±0.14 | 7.84 | 70.12 | 424.35 |
| SLTT [58] | ✗ | ✓ | 95.07±0.04 | 4.88 (↓ 3.2×) | 78.89±0.16 | 12.38 (↓ 3.2×) | 96.61±0.06 | 2.54 (↓ 3.1×) | 70.11 | 144.37 (↓ 2.9×) |
| DECOLLE [25] | ✓ | ✓ | 62.05±0.63 | 2.76 (↓ 5.6×) | 33.89±0.39 | 6.07 (↓ 6.5×) | 79.04±0.31 | 1.81 (↓ 4.3×) | 16.26 | 86.33 (↓ 4.9×) |
| ELL [18] | ✓ | ✓ | 88.40±0.13 | 2.76 (↓ 5.6×) | 66.48±0.24 | 6.07 (↓ 6.5×) | 94.79±0.03 | 1.81 (↓ 4.3×) | 48.99 | 87.83 (↓ 4.8×) |
| STDL | ✓ | ✓ | 94.99±0.13 | 3.51 (↓ 4.4×) | 78.91±0.11 | 8.31 (↓ 4.7×) | 96.71±0.07 | 1.82 (↓ 4.3×) | 69.87 | 106.53 (↓ 4.0×) |

the memory footprint without sacrificing accuracy. In the following, we detail how STDL computes gradients for a given layer $j$ in the $k$-th subnetwork and its auxiliary network.

To explicitly illustrate the presence of temporally dependent gradients in BPTT, we expand the gradient expression in Eq. (6) by unfolding the term $\frac{\partial \mathcal{L}^k}{\partial \boldsymbol{m}^\ell[t+1]}\frac{\partial \boldsymbol{m}^\ell[t+1]}{\partial \boldsymbol{m}^\ell[t]}$ along the temporal dimension of the next layer. This yields the following gradient expression for the membrane potential $\boldsymbol{m}^j[t]$ in layer $j$ of subnetwork $k$:

$$\frac{\partial \mathcal{L}^k}{\partial \boldsymbol{m}^j[t]} = \frac{\partial \mathcal{L}^k}{\partial \boldsymbol{m}^{j+1}[t]}\frac{\partial \boldsymbol{m}^{j+1}[t]}{\partial \boldsymbol{m}^j[t]} + \underbrace{\sum_{t'=t+1}^{T} \frac{\partial \mathcal{L}^k}{\partial \boldsymbol{m}^{j+1}[t']}\frac{\partial \boldsymbol{m}^{j+1}[t']}{\partial \boldsymbol{m}^j[t']}\frac{\partial \boldsymbol{m}^j[t']}{\partial \boldsymbol{m}^j[t]}}_{\text{Temporally Dependent Gradients}}. \tag{16}$$

Eq. (16) reveals that the gradient at the current time step $t$ is influenced by a summation over all future time steps $t' > t$. These temporally dependent gradients arise primarily from the term $\frac{\partial \boldsymbol{m}^j[t']}{\partial \boldsymbol{m}^j[t]}$, which reflects the backward temporal influence from future states $t'$ back to the current time $t$. When omitting the reset process, as is common in prior studies [26], [39], this term can be approximated as $\frac{\partial \boldsymbol{m}^j[t']}{\partial \boldsymbol{m}^j[t]} \approx \lambda^{t'-t}$, where $\lambda$ is the decay factor of the membrane potential. As the temporal gap $t'-t$ increases, this term decays exponentially, diminishing its contribution to the overall gradient.

To enable gradient computation at the current time step $t$, we discard temporally dependent gradients, as their contribution to the overall gradient is negligible. Formally, we define $\boldsymbol{\psi}^j[t]$ as the total influence of $\boldsymbol{m}^j[t]$ on the local instantaneous loss $\mathcal{L}^k[t]$, and it can be calculated recursively as:

$$\boldsymbol{\psi}^j[t] = \begin{cases} \frac{\partial \mathcal{L}^k[t]}{\partial \boldsymbol{m}^J[t]}, & j = J \\ \boldsymbol{\psi}^{j+1}[t]\frac{\partial \boldsymbol{m}^{j+1}[t]}{\partial \boldsymbol{m}^j[t]}, & j < J \end{cases} \tag{17}$$

where $J$ represents the last layer of the auxiliary network. Using this quantity, the gradient of the local loss with respect to the weights of layer $j$ is computed as:

$$\frac{\partial \mathcal{L}^k}{\partial \boldsymbol{W}^j} = \sum_{t=1}^{T} \boldsymbol{\psi}^j[t]\frac{\partial \boldsymbol{m}^j[t]}{\partial \boldsymbol{W}^j} = \sum_{t=1}^{T} \boldsymbol{\psi}^j[t]\boldsymbol{s}^{j-1}[t]^\top. \tag{18}$$

Both $\boldsymbol{\psi}^j[t]$ and $\boldsymbol{s}^{j-1}[t]$ in Eq. (18) are readily available at time step $t$, thereby allowing the online learning of each subnetwork and its auxiliary network. This effectively decouples temporal dependencies and ensures that the memory footprint is independent of the total number of time steps.

TABLE III
COMPARISON OF STDL AND BPTT-BASED METHODS WITH THE ACCURACY RESULTS SOURCED FROM THEIR ORIGINAL PAPERS.

| Dataset | Method | Network | T | Accuracy (%) |
|---|---|---|---|---|
| CIFAR-10 | LTL-Online [53] | ResNet-20 | 16 | 93.15 |
| | Dspike [34] | ResNet-20 | 6 | 94.25±0.07 |
| | GLIF [42] | ResNet-18 | 4 | 94.67±0.05 |
| | **STDL** | **ResNet-18** | **4** | **94.99±0.13** |
| | TET [36] | ResNet-19 | 4 | 94.44±0.08 |
| | RecDis [37] | ResNet-19 | 4 | 95.53±0.05 |
| | **STDL** | **ResNet-19** | **4** | **96.15±0.07** |
| CIFAR-100 | Dspike [34] | ResNet-18 | 4 | 73.35±0.14 |
| | RecDis [37] | ResNet-18 | 4 | 74.10±0.13 |
| | TET [36] | ResNet-18 | 6 | 74.72±0.28 |
| | **STDL** | **ResNet-18** | **4** | **75.35±0.22** |
| | RecDis [37] | ResNet-19 | 4 | 74.10±0.13 |
| | TET [36] | ResNet-19 | 4 | 74.47±0.15 |
| | **STDL** | **ResNet-19** | **4** | **78.91±0.11** |
| ImageNet | PLIF [39] | SEWResNet-34 | 4 | 67.04 |
| | GLIF [42] | ResNet-34 | 4 | 67.52 |
| | TET [36] | SEWResNet-34 | 4 | 68.00 |
| | TEBN [38] | SEWResNet-34 | 4 | 68.28 |
| | **STDL** | **SEWResNet-34** | **4** | **69.87** |

## V. EXPERIMENTS

We evaluate STDL on image classification (Section V-A) and event-based vision datasets (Section V-B). Then, we present ablation studies and analytical results to provide a deeper understanding of STDL (Sections V-C–V-G). Finally, we examine its generalization ability to different neurons, architectures, and temporal datasets (Section V-H–V-I).

### A. Performance Evaluation on Image Classification Datasets

*1) Experiment Setup:* **Datasets.** We use four public datasets for the experiments on image classification, including CIFAR-10 [28], CIFAR-100 [28], SVHN [29], and ImageNet [30]. Following [34]–[36], we apply the autoaugment and cutout data augmentation techniques to CIFAR-10 and CIFAR-100 and employ standard data augmentation techniques on SVHN and ImageNet. More details of the datasets can be found in Appendix C. In line with previous works [18], [39], [58], we convert images into temporal sequences by replicating the image across a given number of time steps. These sequences are then directly fed into SNNs, with the first layer serving as an encoding layer that converts pixel values into spikes.

**Baselines.** We compare STDL with four learning methods, including BPTT [14], SLTT [58], DECOLLE [25], and ELL [18]. They have different decoupled dimensions as shown in Table II. Specifically, DECOLLE and ELL are local learning methods that decouple both spatial and temporal dependencies; SLTT is an online learning method that only decouples temporal dependencies; and BPTT is an end-to-end training method without decoupling. For fair comparisons, we re-implement these baselines and use consistent training settings.

**Training Configurations.** We adopt different spiking architectures, including ResNet-18 [27], ResNet-19 [27], VGG16, and SEWResNet-34 [26]. To apply STDL under a given memory constraint, we first ensure that each auxiliary network contains at least one layer in addition to the classifier layer, and then construct subnetworks within the remaining memory budget, followed by the construction of their corresponding auxiliary networks. Detailed configurations of subnetworks and auxiliary networks are provided in Appendix E. Note that we treat the minimal indivisible unit of a network as a single layer, which corresponds to a residual block and a convolutional layer in ResNets and VGGs, respectively. More training configurations are provided in Appendix D.

*2) STDL Achieves Comparable Accuracy to BPTT with Substantial Memory Savings:* Table II summarizes the performance of the proposed STDL method in comparison to the baselines. STDL consistently achieves accuracy comparable to or better than that of BPTT, while significantly saving the GPU memory footprint for training by $4.0\times$-$4.7\times$. This superior accuracy and memory advantage hold across various datasets and architectures, highlighting the strong generalizability of STDL. Compared to SLTT, STDL achieves approximately 30% lower GPU memory consumption while maintaining comparable accuracy. This demonstrates that STDL's spatial decoupling strategy effectively improves memory efficiency without compromising accuracy.

*3) STDL Significantly Outperforms Local Learning Methods:* The accuracy gains of STDL are especially pronounced when compared to existing local learning methods such as DECOLLE and ELL. For example, on CIFAR-100 with ResNet-19, STDL improves accuracy by over 45% and 12% compared to DECOLLE and ELL, respectively. These results clearly demonstrate the effectiveness of STDL in addressing the limitations of local learning.

*4) STDL Effectively Scales to Large-Scale Datasets:* To validate the scalability of STDL with a growing amount of training data, we conduct experiments on the challenging ImageNet dataset using SEWResNet-34 [26]. The results are shown in Table II. It can be observed that local learning methods struggle to perform well on this large-scale dataset. In contrast, STDL achieves competitive accuracy to BPTT, and outperforms DECOLLE by over 50% and ELL by over 20%. Moreover, STDL significantly reduces the GPU memory usage during training by $4.0\times$ compared to BPTT while maintaining comparable accuracy. These results clearly demonstrate the scalability of STDL in handling large-scale datasets.

*5) STDL Achieves Competitive Accuracy Compared to BPTT-Based Methods:* We compare STDL with BPTT-based training methods, including surrogate gradients [34], loss functions [36], [37], neuron models [39], [42], and normalization methods [38]. As shown in Table III, STDL consistently achieves competitive accuracy across different architectures and datasets. This reaffirms the superior accuracy of STDL.

TABLE IV
COMPARISON OF WALL-CLOCK TRAINING TIME IN HOURS AMONG DIFFERENT LEARNING METHODS. NUMBERS IN PARENTHESES DENOTE THE RELATIVE CHANGE IN TRAINING TIME WITH RESPECT TO BPTT.

| Dataset | CIFAR-10 | CIFAR-100 | SVHN | ImageNet |
|---|---|---|---|---|
| Network | ResNet-18 | ResNet-19 | VGG16 | SEWResNet-34 |
| T | 4 | 4 | 4 | 4 |
| Method | Time/Task | Time/Task | Time/Task | Time/Epoch |
| BPTT | 6.6h | 18.6h | 2.8h | 3.9h |
| DECOLLE [25] | 6.3h (↓5%) | 17.3h (↓7%) | 2.7h (↓4%) | 3.8h (↓3%) |
| ELL [18] | 6.3h (↓5%) | 17.4h (↓6%) | 2.7h (↓4%) | 3.8h (↓3%) |
| STDL | 8.1h (↑23%) | 21.8h (↑17%) | 3.0h (↑7%) | 4.4h (↑13%) |

*6) Computational Overhead of STDL:* The auxiliary networks used in STDL introduce computational overhead during training. To quantify it, we compare the wall-clock training time among different learning methods. As shown in Table IV, STDL incurs a modest increase in training time across all evaluated datasets, with an overhead of approximately 10%–20% relative to BPTT. Despite this additional cost, STDL significantly reduces GPU memory consumption compared to BPTT and achieves substantially higher accuracy than DECOLLE and ELL. Consequently, STDL offers a favorable trade-off between computational cost, memory efficiency, and accuracy, making it well-suited for training large-scale SNNs under limited memory budgets. Notably, the computational overhead of STDL is limited to the training stage, and no overhead is introduced during inference, as the auxiliary networks are removed after training. Compared with DECOLLE and ELL, STDL incurs a computational overhead of up to 29% across all evaluated datasets. Considering the substantial accuracy gains achieved by STDL, this additional computational overhead is modest and represents a favorable trade-off.

*7) Analysis of Auxiliary Networks in STDL:* The modest computational overhead of STDL is mainly attributed to the fact that the auxiliary networks introduced for each subnetwork remain lightweight. Here, we provide an analysis of the auxiliary networks constructed by STDL. As detailed in Appendix E, the average depths of the auxiliary networks for ResNet-18, ResNet-19, VGG16, and SEWResNet-34 are $3.5$, $2.5$, $2.0$, and $4.0$ layers, respectively. These results indicate that STDL does not rely on very deep auxiliary networks to guide subnetwork training. Beyond the average depth statistics, we further observe a clear hierarchical pattern in the auxiliary networks. Specifically, auxiliary networks associated with earlier subnetworks tend to be deeper than those attached to later subnetworks along the network hierarchy. This is expected as earlier subnetworks are followed by more remaining layers and therefore require deeper auxiliary networks to approximate the subsequent transformations.

### *B. Performance Evaluation on Event-Based Vision Datasets*

*1) Experiment Setup:* **Datasets.** We further evaluate the performance of STDL on three event-based vision recognition datasets, including CIFAR10-DVS [31], GAIT-DAY-DVS [32],

TABLE V
COMPARISON OF STDL AND BASELINES ON THREE EVENT-BASED VISION DATASETS. THE GPU MEMORY IS MEASURED WITH A BATCH SIZE OF 100.

| Dataset | Method | T | Accuracy (%) | GPU Mem. (GB) |
|---|---|---|---|---|
| CIFAR10-DVS | Dspike [34] | 10 | 75.40±0.05 | - |
| | OTTT [57] | 10 | 76.27±0.05 | - |
| | TET [36] | 10 | 77.33±0.21 | - |
| | BPTT [14] | 10 | 77.87±0.38 | 99.09 |
| | SLTT [58] | 10 | 77.03±0.19 | 13.08 (↓ 7.6×) |
| | DECOLLE [25] | 10 | 61.40±0.51 | 10.03 (↓ 9.9×) |
| | ELL [18] | 10 | 72.00±0.33 | 10.03 (↓ 9.9×) |
| | STDL | 10 | 77.87±0.19 | 10.50 (↓ 9.4×) |
| GAIT-DAY-DVS | EV-Gait GCN [32] | 1 | 89.9 | - |
| | ASA [68] | 60 | 93.6 | - |
| | BPTT [14] | 20 | 94.15±0.51 | 227.36 |
| | SLTT [58] | 20 | 93.88±0.57 | 15.49 (↓ 14.7×) |
| | DECOLLE [25] | 20 | 84.28±0.88 | 8.89 (↓ 25.6×) |
| | ELL [18] | 20 | 85.22±0.87 | 8.89 (↓ 25.6×) |
| | STDL | 20 | 94.40±0.58 | 10.91 (↓ 20.8×) |
| HAR-DVS | SlowFast [33] | - | 46.54 | - |
| | ACTION-Net [33] | - | 46.85 | - |
| | ASA [68] | 8 | 47.10 | - |
| | BPTT [14] | 4 | 48.64 | 51.20 |
| | SLTT [58] | 4 | 48.01 | 15.86 (↓ 3.2×) |
| | DECOLLE [25] | 4 | 32.66 | 12.08 (↓ 4.2×) |
| | ELL [18] | 4 | 40.51 | 12.10 (↓ 4.2×) |
| | STDL | 4 | 48.46 | 12.87 (↓ 4.0×) |

and HAR-DVS [33]. Both CIFAR10-DVS and GAIT-DAY-DVS have the same spatial resolution of $128 \times 128$. HAR-DVS is an event-based human activity recognition (HAR) dataset with a spatial resolution of $346 \times 260$. It is currently the largest event-based HAR dataset, containing 300 activity classes and a total of $107,646$ samples. Fig. 5 visualizes randomly selected samples in the three datasets.

**Training Configurations.** We use VGG-11 on CIFAR10-DVS and ResNet-18 [27] on GAIT-DAY-DVS and HAR-DVS. We train them with the four baselines and STDL under consistent training settings, which are detailed in Appendix D. For training ResNet-18 with STDL, we adopt the same configuration of subnetworks and auxiliary networks as that in the previous experiments to examine its generalizability.

*2) STDL Consistently Achieves Superior Performance:* Table V presents the results of the proposed STDL method as well as other baseline methods. STDL consistently achieves comparable or even better accuracy when compared to BPTT, SLTT, and other competitive methods across the three datasets. Moreover, STDL offers a significant saving in GPU memory consumption compared to BPTT, with the reduction percentage increasing as the number of time steps increases. For instance, on CIFAR10-DVS with 10 time steps, STDL achieves a memory reduction of $9.4\times$ compared to BPTT, while on GAIT-DAY-DVS with 20 time steps, the reduction reaches an impressive $20.8\times$. Additionally, STDL significantly outperforms DECOLLE and ELL in accuracy across these datasets. These observations align with those from the image classification experiments in Section V-A, demonstrating the STDL's advantages in high accuracy and memory efficiency. The consistent superior performance of our STDL method across various datasets verifies its generalizability and effectiveness in different vision applications.

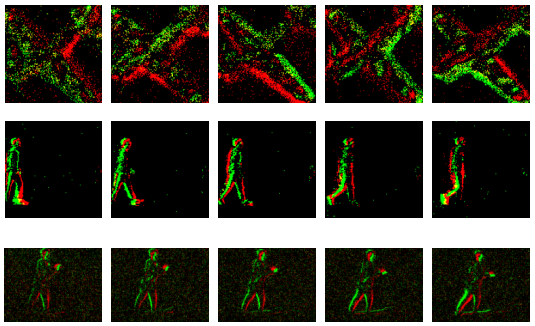


Fig. 5. **Visualization of event streams** (accumulated over 10ms) in CIFAR10-DVS [31], GAIT-DAY-DVS [32], and HAR-DVS [33] from top to bottom.

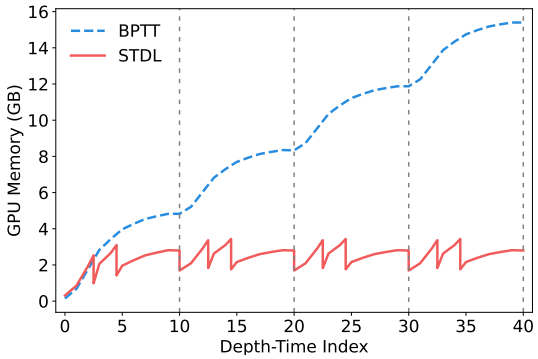



Fig. 6. **GPU memory usage pattern with respect to the layer and time index.** ResNet-18 is trained with BPTT and STDL on CIFAR-10 with 4 steps.

### C. GPU Memory Usage Analysis

We compare the GPU memory trace of STDL with that of BPTT using ResNet-18 on CIFAR-10 with 4 time steps. As shown in Fig. 6, BPTT exhibits a gradual increase in GPU memory usage as the network layer and time step grow. This is because BPTT performs gradient computation after the full forward propagation is completed, leading to the accumulation of neuronal states in memory. In comparison, STDL shows fluctuating GPU memory usage that peaks at a middle layer due to its local updates, which allows cached states to be freed up. Notably, the non-linear increase in GPU memory usage as the network depth grows reflects the decreasing memory footprint of the layers as they approach the output layer in ResNet-18. This demonstrates the rationality of STDL in combining several successive layers into a single subnetwork without affecting the peak memory footprint. The visualization of GPU memory evolution provides a compelling explanation of the memory efficiency advantage of STDL.

### D. Representation Similarity Analysis

In this part, we analyze the layer-wise representations learned by STDL and compare them with those obtained through BPTT training.

*1) Representation Similarity:* We use centered kernel alignment (CKA) [69] to quantitatively analyze the similarity in the representations of network layers between BPTT and other learning rules. The spike-based representation for a layer is obtained by computing its firing rate. We calculate CKA for each layer and an average CKA score across all layers. The

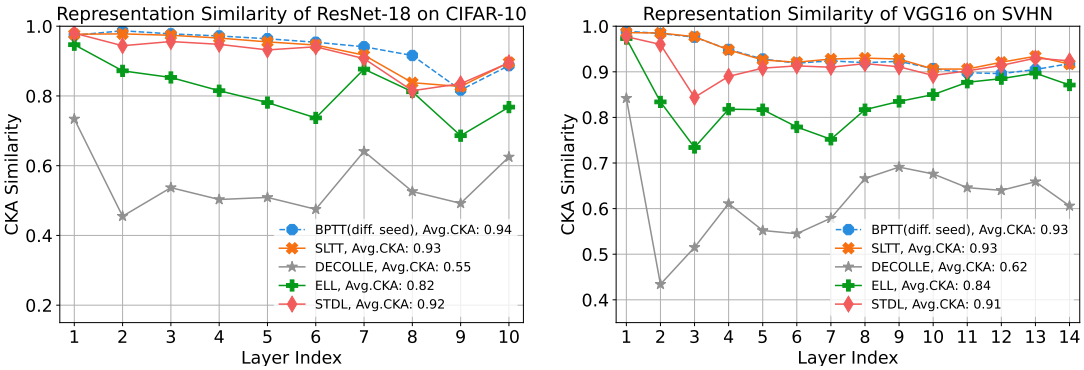



Fig. 7. **Comparison of layer-wise representation similarity.** We utilize CKA [69] to measure the layer-wise similarity of representations between BPTT and other learning rules. To provide a fair baseline for BPTT, we measure the similarity between two networks trained with different random seeds. The average CKA similarity scores over all layers for different learning rules are provided in the legend.

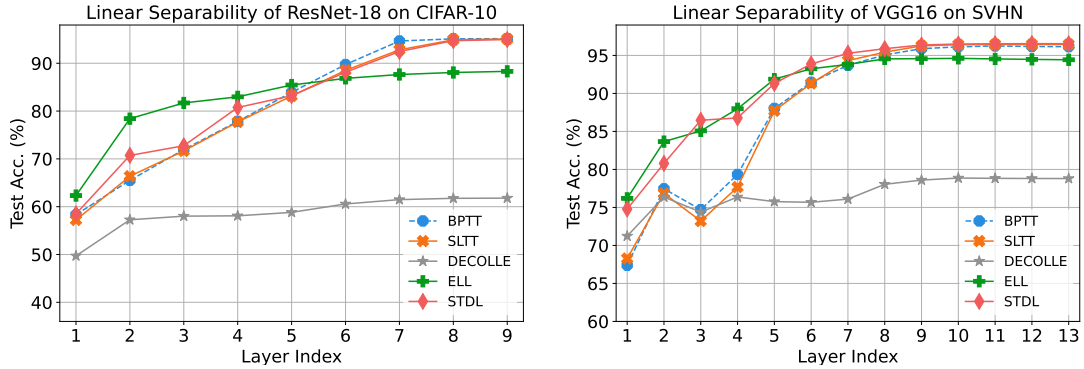



Fig. 8. **Comparison of layer-wise linear separability among STDL and baselines.**

results of ResNet-18 and VGG16 are presented in Fig. 7. The STDL-trained layers generate representations that closely resemble those produced by BPTT, while DECOLLE and ELL show significantly lower representation similarity to BPTT. This offers a compelling explanation for the efficacy of STDL.

*2) Linear Probing:* We further employ the linear probing technique to analyze the layer-wise linear separability of STDL as well as other learning methods. Specifically, we freeze the parameters of well-trained networks and further train additional linear classifiers that are attached to each hidden layer. The results are provided in Fig. 8.

We observe that STDL closely follows BPTT in terms of layer-wise linear separability, whereas DECOLLE and ELL exhibit drastically different patterns compared to BPTT. Specifically, DECOLLE shows consistently lower linear separability across all layers, indicating that its random and fixed local classifiers are insufficient for guiding hidden layers to produce meaningful representations. ELL, on the other hand, encounters the weak coupling issue, where the early layers exhibit greater linear separability than their BPTT counterparts, but this separability significantly deteriorates in layers closer to the output. In comparison, STDL initially shows lower linear separability in the early layers, but exhibits substantial improvements in the middle and output layers, closely resembling the pattern observed in BPTT. These findings confirm that STDL effectively addresses the weak coupling issue and achieves layer-wise representations comparable to those trained by BPTT. This alignment underpins the comparable accuracy between STDL and BPTT.

### *E. Influences of Auxiliary Networks and Subnetworks*

We conduct ablation studies to examine the individual contributions of the two core components in STDL: the auxiliary network construction strategy and the greedy network partitioning method. All experiments are performed on CIFAR-10 using ResNet-18, and the results are summarized in Fig. 9.

We begin by analyzing the design of auxiliary networks in STDL, which incorporates two key principles: reusing the structure of subsequent layers, and maximizing expressive capacity within the memory budget. To assess the role of structural reuse, we replace the residual blocks in the auxiliary networks with either $3\times3$ or $1\times1$ convolutional layers. Both modifications lead to substantial drops in accuracy, confirming that generic convolutions fail to provide structure priors. We further evaluate a variant that repeats the last layer of each subnetwork to match its target depth. Although this variant preserves the depth of the auxiliary network, its accuracy drops from 94.99% to 94.56% compared to STDL. This suggests that increasing depth alone, without exploiting the heterogeneous structure of subsequent layers, is insufficient. These results collectively underscore the importance of leveraging the architecture of subsequent layers to guide subnetwork training effectively.

Next, we evaluate the importance of maximizing the expressive capacity of auxiliary networks. We compare against a

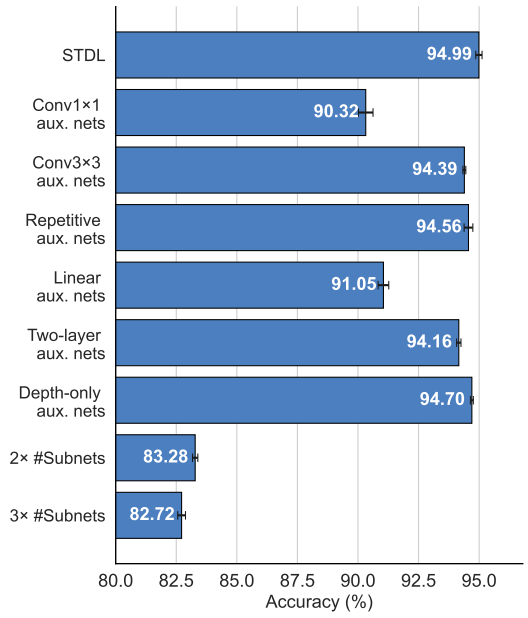



Fig. 9. **Influences of auxiliary networks and subnetworks.** Each row denotes a variant of STDL with a modification described on the y-axis. The blue bar chart reports each variant's accuracy for ResNet-18 on CIFAR-10.

TABLE VI
COMPARISON OF ACCURACY AND GPU MEMORY USAGE BETWEEN STDL AND BPTT WITH VARYING TIME STEPS ON CIFAR-10 USING RESNET-18.

| Method | Metric | T=1 | T=2 | T=4 | T=6 |
|---|---|---|---|---|---|
| BPTT | Mem. (GB) | 4.56 | 8.40 | 15.45 | 22.50 |
| | Acc. (%) | 94.01±0.22 | 94.98±0.17 | 95.12±0.15 | 95.20±0.11 |
| STDL | Mem. (GB) | 3.17 | 3.51 | 3.51 | 3.51 |
| | Acc. (%) | 93.90±0.26 | 94.94±0.14 | 94.99±0.13 | 95.18±0.08 |

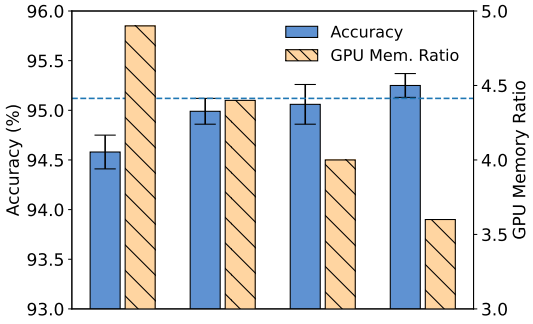



Fig. 10. **Trade-offs between GPU memory efficiency and accuracy in STDL.** The dashed line denotes BPTT's accuracy. GPU memory ratio is the memory of BPTT divided by that of STDL.

simplified variant that uses only a two-layer auxiliary network (i.e., a single layer plus the classifier). This setup results in an accuracy drop of approximately 1%. The performance degrades further when using a linear classifier as the auxiliary network. Another variant that maximizes only the depth also underperforms. These results demonstrate that the expressive capacity of auxiliary networks, in both depth and width, is critical for superior accuracy.

We then evaluate the impact of subnetwork partitioning strategies. Specifically, starting from the configuration of subnetworks and auxiliary networks produced by our STDL method, we construct two variants by adding extra subnetworks to increase the total number of subnetworks by factors of $2\times$ and $3\times$, respectively. The newly added subnetworks are uniformly distributed along the network depth, and each is paired with a linear auxiliary classifier. These variants incur significant accuracy drops due to weakened coupling among subnetworks. These results highlight that minimizing the number of subnetworks under a given memory budget, as achieved by our greedy partitioning strategy, is essential for maintaining strong inter-layer coordination and high accuracy.

### *F. Influence of Varying Time Steps*

We investigate the influence of different numbers of time steps on the performance of STDL. We adopt ResNet-18 on CIFAR-10 and vary the number of time steps from 1 to 6. We compare STDL against BPTT in terms of accuracy and GPU memory usage. Table VI shows that STDL consistently achieves comparable accuracy to BPTT across different time steps. Additionally, unlike BPTT, whose GPU memory footprint linearly increases over time, STDL exhibits a consistent memory footprint over time owing to its temporal decoupling. Furthermore, even with just one time step, STDL demands approximately 30% less GPU memory than BPTT. This reaffirms that the spatial decoupling in STDL can effectively reduce the memory footprint without sacrificing accuracy.

### *G. Trade-Off between GPU Memory Efficiency and Accuracy*

In this part, we investigate the trade-off between GPU memory consumption and model accuracy in STDL. To this end, we vary the level of memory constraint, resulting in different configurations of subnetworks and their corresponding auxiliary networks. Each configuration is then evaluated in terms of accuracy and GPU memory usage, normalized against the memory footprint of BPTT. The results, obtained using ResNet-18 on CIFAR-10, are shown in Fig. 10.

The results reveal that STDL provides a flexible balance between accuracy and memory efficiency. When the memory constraint is relaxed, STDL yields higher accuracy at the cost of increased memory usage. Conversely, tighter memory constraints lead to improved efficiency with only a minor reduction in accuracy. Notably, STDL achieves substantial memory savings even in high-accuracy regimes. For instance, STDL achieves an accuracy of 95.25%, exceeding that of BPTT, while simultaneously reducing GPU memory footprint by a factor of $3.6\times$. This ability to retain competitive performance under different constrained resources highlights the applicability of STDL in memory-constrained scenarios.

### *H. Generalization to Different Neurons and Architectures*

We evaluate the generalization ability of STDL on two spiking neuron models: parametric LIF (PLIF) [39] and adaptive LIF (ALIF) [40]. PLIF introduces trainable decay factors, which necessitate the storage of extra neuronal states during forward propagation. ALIF additionally includes an internal state that evolves over time, which enables adaptive adjustment of neuronal thresholds but also leads to higher GPU memory

TABLE VII
GENERALIZATION OF STDL ON DIFFERENT SPIKING NEURON MODELS AND ARCHITECTURES. THE GAIT-DAY-DVS (T=20) AND CIFAR-100 (T=4) DATASETS ARE ADOPTED.

| Model | Dataset | Method | Accuracy (%) | GPU Mem. (GB) |
|---|---|---|---|---|
| PLIF [39] | GAIT-DAY-DVS | BPTT | 94.50±0.36 | 300.92 |
| | | STDL | 94.80±0.30 | 12.47 ($\downarrow 24.1\times$) |
| ALIF [40] | GAIT-DAY-DVS | BPTT | 94.97±0.32 | 308.48 |
| | | STDL | 95.02±0.19 | 21.42 ($\downarrow 14.4\times$) |
| MSResNet [70] | CIFAR-100 | BPTT | 79.47±0.17 | 41.25 |
| | | STDL | 79.44±0.23 | 8.32 ($\downarrow 5.0\times$) |

TABLE VIII
COMPARISON OF STDL AND BASELINES ON GSC, SHD, AND PS-MNIST.

| Dataset | GSC | SHD | PS-MNIST |
|---|---|---|---|
| Method | T=101 | T=250 | T=784 |
| BPTT [14] | 92.94±0.05 | 85.48±0.71 | 58.25±1.35 |
| SLTT [58] | 88.92±0.17 | 84.72±0.31 | 40.38±0.45 |
| STDL | 89.04±0.07 | 84.65±0.52 | 40.62±0.49 |

consumption. Table VII presents the results on the GAIT-DAY-DVS dataset [32]. Despite the varying memory requirements of these neuron models, STDL consistently achieves significant memory reduction compared to BPTT, without compromising accuracy. Specifically, STDL yields over $24\times$ and $14\times$ memory savings for the PLIF and ALIF models, respectively.

We further evaluate STDL on the MSResNet architecture [70], which introduces membrane-based shortcut connections to facilitate gradient propagation across network depth. As shown in Table VII, STDL achieves accuracy comparable to BPTT while reducing GPU memory consumption by a factor of $5\times$, demonstrating its strong generalization across different architectures. It is worth noting that an advanced variant, Gated Attention Coding MSResNet [71], incorporates attention-based coding to produce more expressive spike representations and thereby achieves higher accuracy. However, the coding method requires pooling operations over features from future time steps. As a result, it is incompatible with online or local learning methods, including STDL. Therefore, we have excluded this model architecture from our evaluation.

These results underscore the strong generalization ability of STDL across different spiking neurons and architectures.

### *I. Evaluation on Tasks with Long Time Steps*

We further evaluate STDL on three datasets with long sequences, including Google Speech Commands (GSC) [72], Spiking Heidelberg Dataset (SHD) [73], and Permuted Sequential MNIST (PS-MNIST). The standard experimental setting [74] is used. The results are summarized in Table VIII.

For GSC and SHD, which mainly contain samples that can be recognized without modeling temporal dependencies [74], both STDL and SLTT perform competitively with BPTT. These results suggest that, for tasks dominated by spatial features, neglecting temporal gradients does not degrade performance. In contrast, for PS-MNIST, where accurate classification critically depends on modeling long-range temporal dependencies [74], both STDL and SLTT exhibit a noticeable performance gap compared to BPTT. This suggests that the omitted temporal gradients are crucial in such temporal datasets. Consequently, STDL primarily benefits tasks dominated by spatial features, whereas its effectiveness on long-range temporal tasks remains limited. Future advances in online learning could be incorporated into the STDL framework to enhance the ability to handle long-range temporal dependencies. Notably, STDL consistently achieves accuracy comparable to that of SLTT across all datasets, reaffirming the effectiveness of the proposed spatial decoupling strategies.

## VI. CONCLUSION

In this work, we proposed STDL, a novel training method that combines the high memory efficiency of local learning with the high accuracy of BPTT for SNNs. STDL achieves this by partitioning the full network into subnetworks, each trained independently with an auxiliary network in an online manner. To enhance the coupling among the subnetworks, we propose the greedy network partitioning algorithm that minimizes the number of subnetworks under a given memory constraint. In addition, we propose an information-theoretically grounded auxiliary network construction method that encourages alignment with the representations learned by BPTT. Extensive experiments across a range of datasets, architectures, and neuron models demonstrate that STDL consistently achieves accuracy comparable to that of BPTT while significantly reducing GPU memory consumption. This work would offer a promising direction for scalable and memory-efficient training of SNNs and opens up new opportunities for training SNNs on resource-constrained neuromorphic hardware.

Despite its superior performance, STDL still relies on gradients computed via backward passes in the subnetwork and its auxiliary network. As a direction for future research, explore the use of forward learning [75] to eliminate the need for the backward pass, which could further reduce the memory footprint during the training process.

## REFERENCES


[1] P. Dayan and L. F. Abbott, *Theoretical neuroscience: computational and mathematical modeling of neural systems*. Cambridge, MA, USA: MIT press, 2001, vol. 806.

[2] R. C. Berwick, A. D. Friederici, N. Chomsky, and J. J. Bolhuis, "Evolution, brain, and the nature of language," *Trends in Cognitive Sciences*, vol. 17, no. 2, pp. 89–98, 2013.

[3] Q. Yu, M. Tsodyks, H. Sompolinsky, D. Schmitz, and R. Gütig, "Interactions between long-and short-term synaptic plasticity transform temporal neural representations into spatial," *Proceedings of the National Academy of Sciences*, vol. 122, no. 47, p. e2426290122, 2025.

[4] B. A. Richards, T. P. Lillicrap, P. Beaudoin, Y. Bengio, R. Bogacz, A. Christensen, C. Clopath, R. P. Costa, A. de Berker, S. Ganguli *et al.*, "A deep learning framework for neuroscience," *Nature Neuroscience*, vol. 22, no. 11, pp. 1761–1770, 2019.

[5] W. Maass, "Networks of spiking neurons: the third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[6] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.

[7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[8] X. Hao, C. Ma, Q. Yang, J. Wu, and K. C. Tan, "Toward ultralow-power neuromorphic speech enhancement with spiking-fullsubnet," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 9, pp. 17 350–17 364, 2025.

[9] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[10] J. Pei, L. Deng, S. Song, M. Zhao, Y. Zhang, S. Wu, G. Wang, Z. Zou, Z. Wu, W. He *et al.*, "Towards artificial general intelligence with hybrid tianjic chip architecture," *Nature*, vol. 572, no. 7767, pp. 106–111, 2019.

[11] G. Li, L. Deng, H. Tang, G. Pan, Y. Tian, K. Roy, and W. Maass, "Brain-inspired computing: A systematic survey and future trends," *Proceedings of the IEEE*, vol. 112, no. 6, pp. 544–584, 2024.

[12] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, 2023.

[13] P. J. Werbos, "Backpropagation through time: what it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.

[14] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-temporal backpropagation for training high-performance spiking neural networks," *Frontiers in Neuroscience*, vol. 12, p. 331, 2018.

[15] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[16] S. Lian, J. Shen, Q. Liu, Z. Wang, R. Yan, and H. Tang, "Learnable surrogate gradient for direct training spiking neural networks," in *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*, 2023, pp. 3002–3010.

[17] F. Zenke and E. O. Neftci, "Brain-inspired learning on neuromorphic substrates," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 935–950, 2021.

[18] C. Ma, R. Yan, Z. Yu, and Q. Yu, "Deep spike learning with local classifiers," *IEEE Transactions on Cybernetics*, vol. 53, no. 5, pp. 3363–3375, 2023.

[19] C. Ma, J. Wu, C. Si, and K. Tan, "Scaling supervised local learning with augmented auxiliary networks," in *The Twelfth International Conference on Learning Representations*, 2024.

[20] G.-q. Bi and M.-m. Poo, "Synaptic modifications in cultured hippocampal neurons: dependence on spike timing, synaptic strength, and postsynaptic cell type," *Journal of Neuroscience*, vol. 18, no. 24, pp. 10 464–10 472, 1998.

[21] P. U. Diehl and M. Cook, "Unsupervised learning of digit recognition using spike-timing-dependent plasticity," *Frontiers in Computational Neuroscience*, vol. 9, p. 99, 2015.

[22] S. R. Kheradpisheh, M. Ganjtabesh, S. J. Thorpe, and T. Masquelier, "STDP-based spiking deep convolutional neural networks for object recognition," *Neural Networks*, vol. 99, pp. 56–67, 2018.

[23] Y. Hao, X. Huang, M. Dong, and B. Xu, "A biologically plausible supervised learning method for spiking neural networks using the symmetric stdp rule," *Neural Networks*, vol. 121, pp. 387–395, 2020.

[24] M. P. E. Apolinario and K. Roy, "S-TLLR: STDP-inspired temporal local learning rule for spiking neural networks," *Transactions on Machine Learning Research*, 2025.

[25] J. Kaiser, H. Mostafa, and E. Neftci, "Synaptic plasticity dynamics for deep continuous local learning (DECOLLE)," *Frontiers in Neuroscience*, vol. 14, p. 424, 2020.

[26] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, "Deep residual learning in spiking neural networks," in *Advances in Neural Information Processing Systems*, vol. 34, 2021, pp. 21 056–21 069.

[27] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.

[28] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," *Master's thesis, Department of Computer Science, University of Toronto*, 2009.

[29] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, A. Y. Ng *et al.*, "Reading digits in natural images with unsupervised feature learning," in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, vol. 2011, no. 5, 2011, p. 7.

[30] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2009, pp. 248–255.

[31] H. Li, H. Liu, X. Ji, G. Li, and L. Shi, "Cifar10-dvs: an event-stream dataset for object classification," *Frontiers in Neuroscience*, vol. 11, p. 244131, 2017.

[32] Y. Wang, X. Zhang, Y. Shen, B. Du, G. Zhao, L. Cui, and H. Wen, "Event-stream representation for human gaits identification using deep neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3436–3449, 2022.

[33] X. Wang, Z. Wu, B. Jiang, Z. Bao, L. Zhu, G. Li, Y. Wang, and Y. Tian, "Hardvs: Revisiting human activity recognition with dynamic vision sensors," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 6, pp. 5615–5623, 2024.

[34] Y. Li, Y. Guo, S. Zhang, S. Deng, Y. Hai, and S. Gu, "Differentiable spike: Rethinking gradient-descent for training spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 23 426–23 439, 2021.

[35] Z. Wang, R. Jiang, S. Lian, R. Yan, and H. Tang, "Adaptive smoothing gradient learning for spiking neural networks," in *International Conference on Machine Learning*, 2023, pp. 35 798–35 816.

[36] S. Deng, Y. Li, S. Zhang, and S. Gu, "Temporal efficient training of spiking neural network via gradient re-weighting," in *International Conference on Learning Representations*, 2021.

[37] Y. Guo, X. Tong, Y. Chen, L. Zhang, X. Liu, Z. Ma, and X. Huang, "Recdis-snn: Rectifying membrane potential distribution for directly training spiking neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 326–335.

[38] C. Duan, J. Ding, S. Chen, Z. Yu, and T. Huang, "Temporal effective batch normalization in spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 34 377–34 390, 2022.

[39] W. Fang, Z. Yu, Y. Chen, T. Masquelier, T. Huang, and Y. Tian, "Incorporating learnable membrane time constant to enhance learning of spiking neural networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 2661–2671.

[40] B. Yin, F. Corradi, and S. M. Bohté, "Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks," *Nature Machine Intelligence*, vol. 3, no. 10, pp. 905–913, 2021.

[41] Q. Bai, H. Wang, and Q. Yu, "A brain-inspired gating mechanism unlocks robust computation in spiking neural networks," in *The Fourteenth International Conference on Learning Representations*, 2026.

[42] X. Yao, F. Li, Z. Mo, and J. Cheng, "Glif: A unified gated leaky integrate-and-fire neuron for spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 32 160–32 171, 2022.

[43] L. Qin, Z. Wang, R. Yan, and H. Tang, "Attention-based deep spiking neural networks for temporal credit assignment problems," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, no. 8, pp. 10 301–10 311, 2024.

[44] Z. Deng, Z. Liu, J. Wang, S. Chen, X. Wei, and Q. Yu, "Hetsyn: Versatile timescale integration in spiking neural networks via heterogeneous synapses," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[45] P. Li, Z. Ma, Y. Zang, and Q. Yu, "Multi-synaptic cooperation: A bio-inspired framework for robust and scalable continual learning," in *The Fourteenth International Conference on Learning Representations*, 2026.

[46] X. Chen, Q. Yang, J. Wu, H. Li, and K. C. Tan, "A hybrid neural coding approach for pattern recognition with spiking neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 5, pp. 3064–3078, 2024.

[47] Q. Yu, S. Song, C. Ma, L. Pan, and K. C. Tan, "Synaptic learning with augmented spikes," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 3, pp. 1134–1146, 2022.

[48] Q. Yu, C. Ma, S. Song, G. Zhang, J. Dang, and K. C. Tan, "Constructing accurate and efficient deep spiking neural networks with double-threshold and augmented schemes," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 4, pp. 1714–1726, 2022.

[49] Y. Yan, Q. Yang, Y. Wu, H. Liu, M. Zhang, H. Li, K. C. Tan, and J. Wu, "Efficient and robust temporal processing with neural oscillations modulated spiking neural networks," *Nature Communications*, vol. 16, no. 1, p. 8651, 2025.

[50] S. Zhang, Q. Yang, C. Ma, J. Wu, H. Li, and K. C. Tan, "Tc-lif: A two-compartment spiking neuron model for long-term sequential modelling," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 15, 2024, pp. 16 838–16 847.

[51] X. Chen, J. Wu, C. Ma, Y. Yan, Y. Wu, and K. C. Tan, "Pmsn: A parallel multi-compartment spiking neuron for multi-scale temporal processing," 2024. [Online]. Available: https://arxiv.org/abs/2408.14917

[52] A. G. Ororbia, "Brain-inspired machine intelligence: A survey of neurobiologically-plausible credit assignment," 2023. [Online]. Available: https://arxiv.org/abs/2312.09257

[53] Q. Yang, J. Wu, M. Zhang, Y. Chua, X. Wang, and H. Li, "Training spiking neural networks with local tandem learning," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 12 662–12 676.

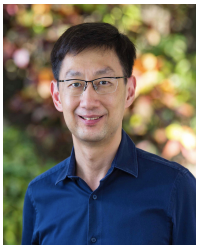
[54] Y. Wu, R. Zhao, J. Zhu, F. Chen, M. Xu, G. Li, S. Song, L. Deng, G. Wang, H. Zheng *et al.*, "Brain-inspired global-local learning incorporated with neuromorphic computing," *Nature Communications*, vol. 13, no. 1, p. 65, 2022.
[55] P. He, R. Xiao, C. Tang, S. Huang, J. Lv, and H. Tang, "Stsf: Spiking time sparse feedback learning for spiking neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 6, pp. 11 479–11 492, 2025.
[56] G. Bellec, F. Scherr, A. Subramoney, E. Hajek, D. Salaj, R. Legenstein, and W. Maass, "A solution to the learning dilemma for recurrent networks of spiking neurons," *Nature Communications*, vol. 11, p. 3625, 2020.
[57] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, "Online training through time for spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 717–20 730, 2022.
[58] Q. Meng, M. Xiao, S. Yan, Y. Wang, Z. Lin, and Z.-Q. Luo, "Towards memory-and time-efficient backpropagation for training spiking neural networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 6166–6176.
[59] B. Yin, F. Corradi, and S. M. Bohté, "Accurate online training of dynamical spiking neural networks through forward propagation through time," *Nature Machine Intelligence*, vol. 5, no. 5, pp. 518–527, 2023.
[60] Z. Wang, Y. Zhang, S. Lian, X. Cui, R. Yan, and H. Tang, "Toward high-accuracy and low-latency spiking neural networks with two-stage optimization," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 2, pp. 3189–3203, 2025.
[61] J. Wu, Y. Chua, M. Zhang, G. Li, H. Li, and K. C. Tan, "A tandem learning rule for effective training and rapid inference of deep spiking neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 1, pp. 446–460, 2023.
[62] J. Wu, C. Xu, X. Han, D. Zhou, M. Zhang, H. Li, and K. C. Tan, "Progressive tandem learning for pattern recognition with deep spiking neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7824–7840, 2022.
[63] J. Hu, M. Yao, X. Qiu, Y. Chou, Y. Cai, N. Qiao, Y. Tian, B. Xu, and G. Li, "High-performance temporal reversible spiking neural networks with $\mathcal{O}(l)$ training memory and $\mathcal{O}(1)$ inference cost," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, 2024, pp. 19 516–19 530.
[64] M. Yao, X. Qiu, T. Hu, J. Hu, Y. Chou, K. Tian, J. Liao, L. Leng, B. Xu, and G. Li, "Scaling spike-driven transformer with efficient spike firing approximation training," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 4, pp. 2973–2990, 2025.
[65] X. Qiu, M. Zhang, J. Zhang, W. Wei, H. Cao, J. Guo, R.-J. Zhu, Y. Shan, Y. Yang, and H. Li, "Quantized spike-driven transformer," in *The Thirteenth International Conference on Learning Representations*, 2025.
[66] D. O. Hebb, "The organization of behavior: a neuropsycholocigal theory," *A Wiley Book in Clinical Psychology*, vol. 62, p. 78, 1949.
[67] R. Shwartz-Ziv and N. Tishby, "Opening the black box of deep neural networks via information," 2017. [Online]. Available: https://arxiv.org/abs/1703.00810
[68] M. Yao, J. Hu, G. Zhao, Y. Wang, Z. Zhang, B. Xu, and G. Li, "Inherent redundancy in spiking neural networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 16 924–16 934.
[69] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of neural network representations revisited," in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97, 2019, pp. 3519–3529.
[70] Y. Hu, L. Deng, Y. Wu, M. Yao, and G. Li, "Advancing spiking neural networks toward deep residual learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 2, pp. 2353–2367, 2025.
[71] X. Qiu, R.-J. Zhu, Y. Chou, Z. Wang, L.-j. Deng, and G. Li, "Gated attention coding for training high-performance and efficient spiking neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 1, 2024, pp. 601–610.
[72] P. Warden, "Speech commands: A dataset for limited-vocabulary speech recognition," 2018. [Online]. Available: https://arxiv.org/abs/1804.03209
[73] B. Cramer, Y. Stradmann, J. Schemmel, and F. Zenke, "The heidelberg spiking data sets for the systematic evaluation of spiking neural networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 7, pp. 2744–2757, 2022.
[74] C. Ma, X. Chen, Y. Li, Q. Yang, Y. Wu, G. Li, G. Pan, H. Tang, K. C. Tan, and J. Wu, "Spiking neural networks for temporal processing: Status quo and future prospects," 2025. [Online]. Available: https://arxiv.org/abs/2502.09449
[75] G. Hinton, "The forward-forward algorithm: Some preliminary investigations," 2022. [Online]. Available: https://arxiv.org/abs/2212.13345



**Chenxiang Ma** received the M.E. degree in computer science from Tianjin University, Tianjin, China, in 2022. He is currently pursuing the Ph.D. degree with the Department of Data Science and Artificial Intelligence, The Hong Kong Polytechnic University, Hong Kong SAR, China. His current research interests include spiking neural networks, neuromorphic computing, and efficient artificial intelligence.



**Xinyi Chen** received the B.E. and M.E. degrees from the College of Control Science and Engineering, Zhejiang University, Hangzhou, China, in 2019 and 2022, respectively, and the Ph.D. degree from the Department of Data Science and Artificial Intelligence, The Hong Kong Polytechnic University, Hong Kong, China, in 2025. She is currently a Postdoctoral Research Fellow at The Hong Kong Polytechnic University. Her research interests include neuromorphic computing, spiking neural networks, and energy-efficient intelligent systems.



**Kay Chen Tan** (Fellow, IEEE) received the B.Eng. (Hons.) and Ph.D. degrees from the University of Glasgow, Glasgow, U.K., in 1994 and 1997, respectively. He is currently the Head and Chair Professor of Computational Intelligence at the Department of Data Science and Artificial Intelligence, The Hong Kong Polytechnic University, Hong Kong SAR, China. He is also an Honorary Professor with the University of Nottingham in the UK, and Chief CoEditor of Springer Book Series on *Machine Learning: Foundations, Methodologies, and Applications*. Dr. Tan served as the Vice-President (Publications) for the IEEE Computational Intelligence Society, USA, from 2021 to 2024.



**Jibin Wu** (Member, IEEE) received the B.E. and Ph.D degree in Electrical Engineering from National University of Singapore, Singapore in 2016 and 2020, respectively. Dr. Wu is currently an Assistant Professor in the Department of Data Science and Artificial Intelligence, The Hong Kong Polytechnic University. His research interests broadly include brain-inspired artificial intelligence, neuromorphic computing, computational audition, speech processing, and machine learning.

Dr. Wu has published over 70 papers in prestigious conferences and journals in artificial intelligence and speech processing, including Nature Communications, NeurIPS, ICLR, AAAI, TPAMI, TNNLS, TASLP, and IEEE JSTSP. He is currently serving as the Associate Editors for IEEE Transactions on Neural Networks and Learning Systems and IEEE Transactions on Cognitive and Developmental Systems.